3. Logic programming part (Prolog) [8% of your final mark]

For this part of the comprehensive assignment, you have to implement a logic programming solution of the stable matching problem for residents and programs with the iterative Gale-Shapley algorithm. Refer to the general problem description section for the algorithmic description.

The input to your program is in the form of a knowledge base listing the residents and programs information as well as their respective rank order lists (see files `rp.pl` and `rp4000.pl`). In the case of the small version of the problem that you can use for testing and debugging, this knowledge base is as follows:

```
resident(574, name(salvatore,williams), [nrs,hep,mmi]).
resident(517, name(rosalie,frederick), [nrs,mmi]).
resident(126, name(indie,medrano), [mmi,nrs]).
resident(828, name(emma,tremmo), [obg,nrs]).
resident(403, name(aspyn,olson), [hep,nrs]).
resident(226, name(zev,jarvis), [mmi,hep,nrs]).
resident(913, name(camille,paquet), [obg,mmi,hep]).
resident(773, name(marie,clown), [obg]).
resident(616, name(laurent,robert), [obg,mmi,hep,nrs]).
resident(377, name(tom,tan), [mmi,obg]).
program(nrs, "Neurosurgery",4, [574,517,403,828,226,126]).
program(obg, "Obstetrics and Gynecology",3, [616,828,773,913]).
program(mmi, "Microbiology",1, [574,517,226,913,377,126]).
program(hep, "Hematological Pathology",2, [403,574,913,616,226]).
```

The solution should be obtained by calling the high-level predicate `gale_shapley/0` that will display the solution in the prescribed format (using the predicate `write/1`):

```
?- gale_shapley.

medrano,indie,126,nrs,Neurosurgery
frederick,rosalie,517,nrs,Neurosurgery
williams,salvatore,574,nrs,Neurosurgery
robert,laurent,616,obg,Obstetrics and Gynecology
clown,marie,773,obg,Obstetrics and Gynecology
tremmo,emma,828,obg,Obstetrics and Gynecology
jarvis,zev,226,mmi,Microbiology
paquet,camille,913,hep,Hematological Pathology
olson,aspyn,403,hep,Hematological Pathology
tan,tom,377,XXX,NOT_MATCHED
Number of unmatched residents: 1
Number of positions available: 1
true .
```

The following predicate can help you in the display of your solution:

```
writeMatchInfo(ResidentID,ProgramID):-
     resident(ResidentID,name(FN,LN),_),
     program(ProgramID,TT,_,_),write(LN),write(','),
     write(FN),write(','),write(ResidentID),write(','),
     write(ProgramID),write(','),writeln(TT).
```

Example:

```
?- writeMatchInfo(126,nrs).
medrano,indie,126,nrs,Neurosurgery
true.
```

Internally, we ask you to build the solution using the `match` compound term which simply associates a program with a list of matched residents.

```
match(nrs,[126,517,574])
```

The solution then becomes a list of these matches (the match set):

```
[match(nrs,[126,517,574]),match(obg,[616,773,828]),
     match(mmi,[226]),match(hep,[913,403])]
```

The following query produces the solution in its initial state (with no matched residents):

```
?- findall(match(P,[]),program(P,_,_,_),Ms).
Ms =[match(nrs,[]),match(obg,[]),match(mmi,[]),match(hep,[])].
```

Each time a match is offered to a resident, the match set a new updated match set is produced (see predicate `offer/3` described below).

Some predicates

In order to implement the Gale-Shapley algorithm based on the logic programming paradigm, it is best to define several short predicates that will be combined to build the complete solution. In particular, you <u>must</u> define the following predicates:

- `rankInProgram(ResidentID,ProgramID,Rank)` : that computes the rank of a resident in the ROL of a program.

  ```
  ?- rankInProgram(403,nrs,R).
  R = 3 .

  ?- rankInProgram(403,mmi,R).
  false.
  ```

- `leastPreferred(ProgramID,ResidentIDsList, LeastPreferredResidentID,RankOfThisResident)`: that identifies the least preferred resident (the one with the highest ranking) in a list according to the program's ROL.

```
?- leastPreferred(nrs,[403, 517, 226, 828],Rid,Rank).
Rid = 226,
Rank = 5
```

- `matched(ResidentID,ProgramID,Matcheset)`: that verifies if a resident has been matched to a program.

```
?- Ms = [match(nrs, [517]), match(obg, []), match(mmi, [126]),
          match(hep, [226,574])], matched(226,P,Ms).
P = hep.
```

- `offer(ResidentID,currentMatchset,newMatchset)`: that tries to assign a program to the resident. This predicate corresponds to the body of the main loop of the Gale-Shapley algorithm.

```
?- M = [match(nrs, []), match(obg, []), match(mmi, [126]),
          match(hep, [226,574])], offer(517,M, NewM).
NewM = [match(nrs, [517]), match(obg, []),
          match(mmi, [126]), match(hep, [226, 574])].

?- M = [match(nrs, [517]), match(obg, []), match(mmi, [126]),
          match(hep, [226,574])], offer(403,M, NewM).
NewM = [match(nrs, [517]), match(obg, []),
          match(mmi, [126]), match(hep, [403, 574])].
```

You will have to define additional predicates but the list above constitutes the core of your program.

Few important considerations

It is important to make use of the cut (!) operator to reduce the research space and the compute time. Cuts are also useful to implement if-then-else patterns. Note that Prolog is not a very efficient language (it can take several minutes, probably less than 10 however, to obtain the solution to the `rp4000.pl` file) but it is a very expressive language (the full solution, including the predicates that display the solution could contain less than 50 lines of code!).

The `offer/3` predicate is the most important predicate in your program. It tries to match a resident with the programs in its ROL (in order of preference). It would be advised to create a helper predicate that tries to match one resident with one program. The `offer/3` predicate will simply call this helper predicate for all programs of the ROL until a match is found. In case of success, the predicate produces

an updated match set. If the resident cannot be matched, the new match set will be identical to the current one.

Following the Gale Shapley algorithm, you have to go through all the resident by calling the `offer/3` predicate for each of them. One of your predicates will have the responsibility to do this. However, sometimes, when a resident is matched, they bump out a previously matched resident who then becomes unmatched. You therefore will have to go through again all the residents to match these unmatched residents. This will be accomplished by another top-level predicate that will be triggered the loop over the residents until they have all been matched or cannot be matched (having exhausted their ROLs), that is until the resulting match set at the end of the loop remains identical to the match set at the beginning of the loop.

Submission

Your solution must include the `gale_shapley/0` top-level predicate that generates the solution for the solution of the `rp4000.pl` knowledge base previously loaded. Submit your project in a zip file containing the definitions of your predicates; all your Prolog predicates must have a short header describing what the predicate does and its parameters. Include also a text file (`output.txt`) containing the results.